SecurePlate: Project Manual

Welcome to the SecurePlate project! This is a system for automatically detecting license plates, checking them against an "allowed" list, and displaying the access status on a live dashboard.

This manual is broken into four phases to help you understand every part of the project, from its setup to its purpose.

Phase 1: Software, Packages, and Setup

Before you can run the project, you need to have the right software and Python packages installed.

1. Software Needed

- **Python:** The programming language itself. This project was built with Python 3 (e.g., 3.10 or 3.11).
- **A Code Editor:** A program to view and edit the code, like VS Code, PyCharm, or even a simple text editor.
- **A Webcam:** The system needs a camera to see the license plates.

2. Python Packages

This project relies on several powerful, open-source Python libraries:

- **`opencv-python`:** (Used as `cv2`) The core computer vision library. It handles grabbing images from the webcam, finding shapes, and processing images.
- **`easyocr`:** This is the Optical Character Recognition (OCR) "engine." It's the AI model that *reads* the text from the plate image.
- **`pandas`:** A data analysis library. We use it to easily read and search the `allowed_list.csv` file.
- **`numpy`:** A library for numerical operations. It's a key dependency for both OpenCV and EasyOCR.

3. `requirements.txt`

To make installation easy for another developer, you can give them this file. Save the text below as `requirements.txt` in your project folder.

Plaintext

```
opencv-python
easyocr
pandas
numpy
```

A developer can then run `pip install -r requirements.txt` to install everything at once.

Phase 2: How the Project Works (Code Explained)

This project works by passing information between several key files. Here is the data flow:

1. `main.py` captures a video frame from the webcam.
2. The frame is sent to `plate_recognition.py` to find a plate.
3. The plate text (e.g., "KJA123AA") is sent to `database_check.py`.
4. `database_check.py` looks in `allowed_list.csv` and returns a status ("ALLOWED").
5. The plate, status, and owner info are sent to `dashboard_update.py`.
6. `dashboard_update.py` edits the `dashboard.html` file.
7. `main.py` also logs the event in `log.csv` and saves a picture in the `images/` folder.

1. Folders and Files (The "Database" and "Dashboard")

- **`images/` (Folder):** This is where `main.py` saves a picture of the *successfully* detected plate.
- **`allowed_list.csv` (File):** This is your database. It's a simple spreadsheet you can edit in Excel or any text editor. It **must** have these columns:
  - `plate`: The full license plate text (e.g., "ABC123DE").
  - `owner`: The name of the plate's owner.
  - `category`: Any other info (e.g., "Student," "Teacher," "Staff").
- **`dashboard.html` (File):** This is the visual webpage. It uses CSS inside the `<style>` tag to look good.
  - It's designed to auto-refresh every 5 seconds.
  - It has special tags like `<p class="status">` and `<p>Plate: ---</p>` that the Python script looks for.

2. The Python "Engine" (`plate_recognition.py`)

This is the most complex and important file. It's responsible for *finding* and *reading* the plate.

- `easyocr.Reader(['en'])`: This line (at the top) loads the AI model into memory. This is why the program might take a few seconds to start.
- `_is_valid_plate_format(text)`: A helper function that uses a **Regular Expression (`re.match`)** to check if the text matches the Nigerian format (`3 letters, 3 numbers, 2 letters`).
- `_repair_plate(raw)`: A helper function that fixes common OCR mistakes. For example, the AI might see a 'Q' and think it's an 'O', or a '1' and think it's an 'I'. This function corrects those.
- `easyocr_read(image)`: This function's only job is to run the OCR model on the small image crop it's given.
- `crop_from_contour(frame, contour)`: This is the "smart crop" function. It takes the 4 corners of the plate (which might be at an angle) and "un-warps" them to create a flat, rectangular image that is easy for the OCR to read.
- **`read_plate_from_frame(frame)` (The Core):** This function is the heart of the engine. It runs on a single frame and does this:

1. **Grayscale:** Converts the image to black and white.
2. **Filter (`cv2.bilateralFilter`)**: Smooths the image to remove "noise" (like film grain) while keeping the edges of the plate sharp.
3. **Edge Detection (`cv2.Canny`)**: Creates a "sketch" of the image, showing only the edges.
4. **Find Contours (`cv2.findContours`):** Finds all the *shapes* in the "sketch."
5. **Filter Contours:** It loops through all the shapes and throws away any that aren't rectangles (`len(approx) == 4`) or aren't the right *shape* (`aspect_ratio` check).
6. **Crop & Read:** Once it finds a good, plate-shaped rectangle, it sends it to `crop_from_contour` and `easyocr_read` to get the text.

- **`read_plate_aggregated(frame, buffer)` (The "Voting" System):** This is the function that `main.py` actually uses.
  - Video is "flickery." A plate might be read correctly on one frame but misread on the next.
  - This function collects 5 readings in a `buffer` (list). It only returns a plate as "detected" if it gets the **same plate text** at least 2 times.
  - This is why you were using `test_detection.py` (which shows every attempt) and `main.py` (which waits for a confident "vote") and they *behaved* differently.

3. The Other Python Files

- **`database_check.py`:**
  - Its `check_plate(plate_text)` function opens `allowed_list.csv` (using `pandas`).
  - It checks if the `plate_text` exists in the 'plate' column.
  - It returns "ALLOWED" and the owner's info, or "NOT ALLOWED" and no info.
- **`dashboard_update.py`:**
  - Its `update_dashboard(...)` function opens `dashboard.html` as a text file.

- It uses **Regular Expressions (`re.sub`)** to find and replace the status, plate, and owner text.
- It also cleverly adds the `allowed` or `denied` CSS class, which makes the status box turn green or red.

- **`main.py` (The Conductor):**
- This file runs the main `while True:` loop that keeps the program alive.
- It reads a frame from the webcam (`cap.read()`).
- It sends the frame to the "voting" function (`read_plate_aggregated`).
- **Crucially**, it only acts if a *new* plate is found (`if plate_text and plate_text != last_plate:`). This stops it from logging the same car 1,000 times.
- When it finds a new plate, it "conducts" the orchestra:

1. Calls `check_plate()`.
2. Calls `update_dashboard()`.
3. Saves the image (`cv2.imwrite`).
4. Saves the log entry (`df.to_csv('log.csv')`).

- It also displays the live camera feed with `cv2.imshow()`.

Phase 3: How to Run the Program

Here is a step-by-step guide for a new user:

1. **Install Python** on your computer.
2. **Download** the project folder with all the files (e.g., `main.py`, `plate_recognition.py`, etc.).
3. **Open a terminal** or command prompt in that project folder.
4. **Install Packages:** Type `pip install -r requirements.txt` and press Enter. (If you don't have the .txt file, install them one by one: `pip install opencv-python pandas easyocr`).
5. **Edit the Database:** Open `allowed_list.csv` with Excel or Notepad. Add the license plates you want to allow, along with their owners. Save the file.
6. **Run the Program:** In your terminal, type `python main.py` and press Enter.
   - A window will pop up showing your webcam feed.
   - The `easyocr` model will load (this may take 10-20 seconds).
7. **View the Dashboard:** Find the `dashboard.html` file in the folder and double-click it. It will open in your web browser.
8. **Test:** Hold a license plate (one from your `allowed_list.csv`) in front of the camera. Hold it steady for a few seconds.
9. **Watch:** The `dashboard.html` page should update to "ACCESS GRANTED" and turn green.
10. **Quit:** Click on the webcam video window and press the '**q**' key to stop the program.

Phase 4: Project Scope, Constraints, and Impact

1. Scope (What it's designed to do)

This project is a **prototype Access Control System**. Its purpose is to:

- **Identify** vehicles in real-time using a standard webcam.
- **Verify** their identity against a local database.
- **Log** all vehicle movements (both allowed and denied) for later review.
- **Provide** immediate visual feedback to a guard or user via a web dashboard.

2. Constraints (Its Weaknesses)

This system is a great prototype, but it's important to know its limitations:

- **Lighting:** Accuracy depends heavily on good, even lighting. It will likely fail at night, in heavy shadows, or in direct, glaring sunlight.
- **Angle & Speed:** The plate must be relatively flat and facing the camera. It will not work on fast-moving cars or cars at a sharp angle.
- **Performance:** It runs on your computer's CPU. Without a powerful graphics card (GPU), it can be slow.
- **Format:** It is *only* trained to find the specific Nigerian plate format. It will ignore all other plate styles.
- **Security:** The "database" is an unsecured `allowed_list.csv` file. Anyone with access to the computer can add or remove plates. This is *not* a high-security system.

3. Security Impact (How it can be used)

This project is a powerful example of how simple AI can be used for real-world security and automation.

- **Small-Scale Access Control:** It's perfect for a low-security environment. Imagine a school where it automatically opens the gate for teachers' cars ("Staff" category) but alerts the guard for student cars ("Student" category).
- **Parking Management:** It could be used to monitor a small office parking lot, logging which employee cars are present.
- **Automation:** It automates the manual, boring job of a guard having to walk up to every car and check a clipboard, making the process faster and more efficient.